

Arquitetura de Computadores

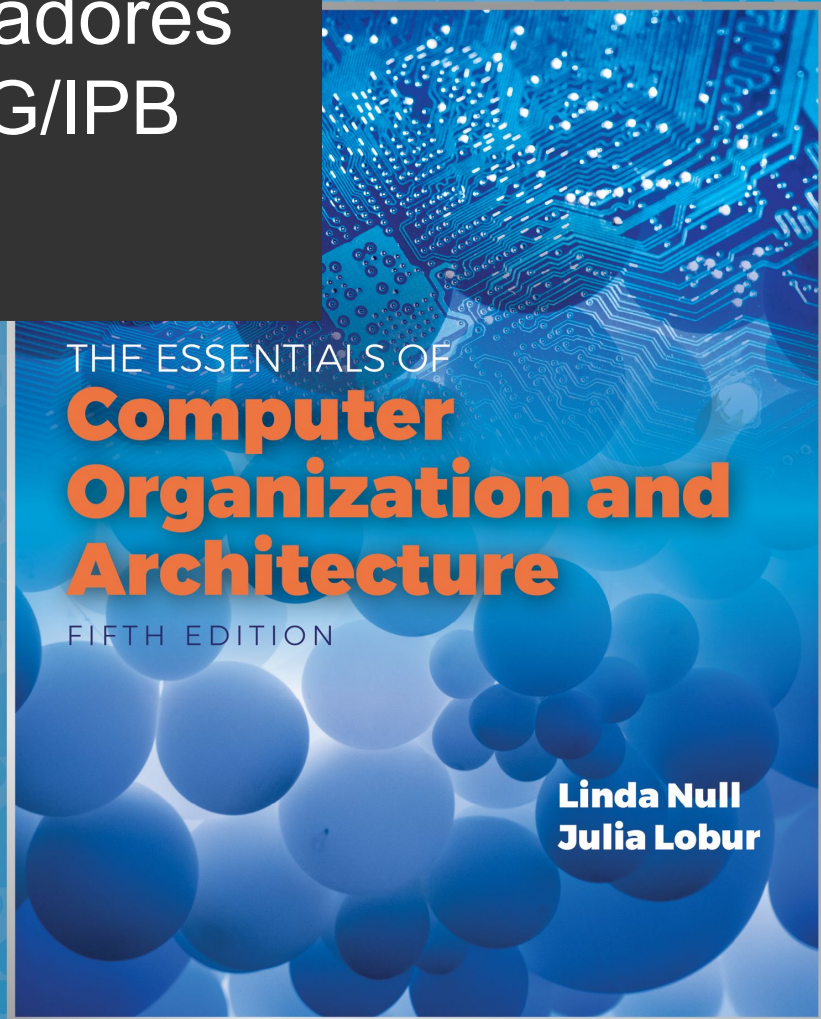
Eng. Informática, ESTiG/IPB

2024/2025

José Rufino

(baseado no Capítulo 2 do livro “The Essentials of Computer Organization and Architecture” de Linda Null e Julia Lobur)

Unidade 2: Representação de Dados



Objetivos

- entender os fundamentos da **representação e manipulação** de dados num sistema de computação
- dominar as **técnicas de conversão** entre diferentes **sistemas de numeração**
- compreender como ocorrem erros de computação devido a **truncagem e overflow**
- entender os conceitos fundamentais da **representação em vírgula flutuante**
- conhecer os principais **códigos de caracteres**

2.1 Introdução



- **bit**: unidade de informação básica num computador
 - **bit** = **b**inary **d**igit
 - corresponde ao estado “on” ou “off” num circuito digital
 - ... ou a níveis “altos” ou “baixos” de tensão (*voltage*)
- **byte**: grupo de 8 bits (IBM System/360, 1964)
 - a mais pequena unidade de armazenamento “endereçável”
 - “endereçável” significa que um byte em particular pode ser acedido (lida/escrita) dada a sua localização em memória

2.1 Introdução

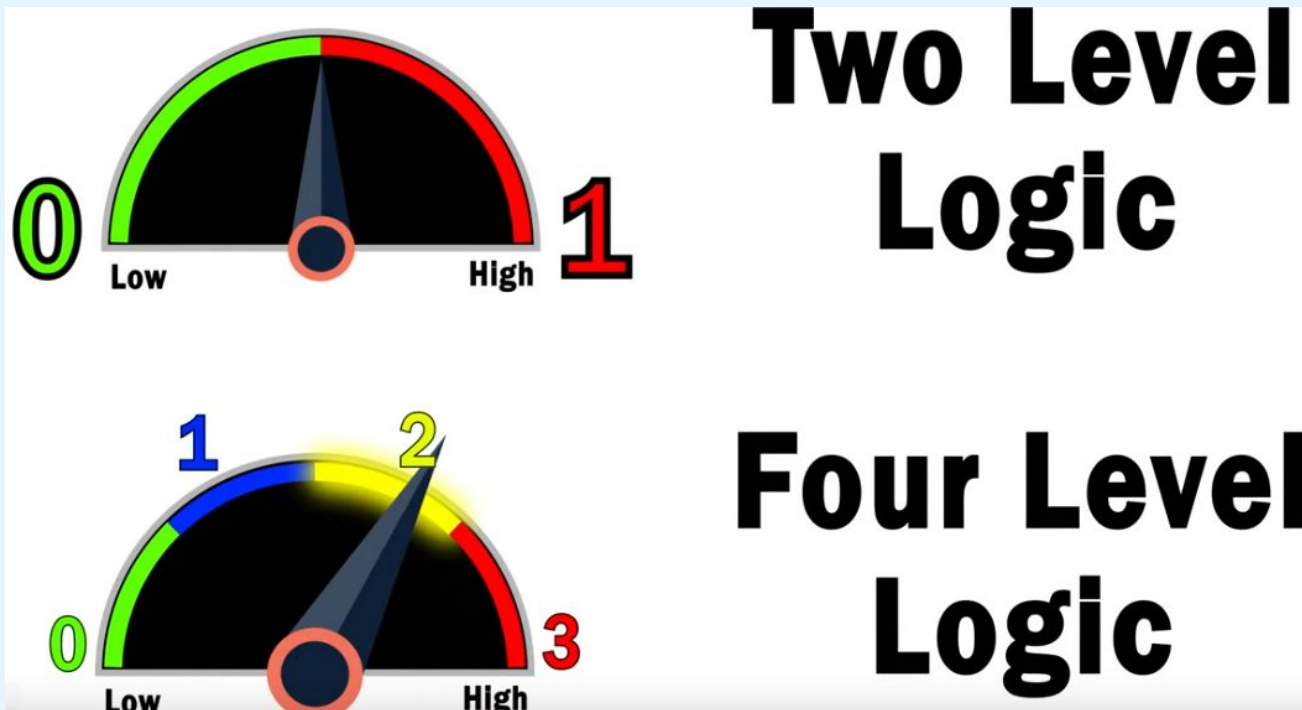


- ***palavra (word)***: grupo contíguo de bytes
 - as *palavras* podem ter qualquer número de bits ou bytes
 - os tamanhos 16, 32 e 64 bits são os mais comuns
 - num sistema “endereçável à palavra”, a palavra é a mais pequena unidade de armazenamento possível de endereçar
- ***nibble***: grupo contíguo de 4 bits
 - os bytes são, portanto, constituídos por 2 *nibbles*:
 - o “**mais significativo**” e o “**menos significativo**”
 - exemplo: o byte 10100101 tem os *nibbles* **1010** e **0101**

2.1 Introdução



- os computadores usam o sistema binário porquê ?



Why can't computers use base 3 instead of binary? Voltage states explained.

<https://www.youtube.com/watch?v=fXwSFhUVFmE>

2.2 Sistemas de Numeração Posicionais

- um **número** é uma sequência de algarismos/dígitos (tb designados por numerais), expresso numa certa **base**
- a **base** define o conjunto (alfabeto) de dígitos admissíveis
 - base 10: 10 dígitos (0,1,2,...,9) - sist. decimal
 - base 2: 2 dígitos (0,1) - sist. binário
 - base 3: 3 dígitos (0,1,2) - sist. ternário
 - base 8: 8 dígitos (0,1,...,7) - sist. octal
 - base 16: 16 dígitos (0,1,...,9,A,B,C,D,E,F) - sist. hexadecimal

2.2 Sistemas de Numeração Posicionais

- os Sistemas de Numeração Posicionais são também conhecidos por Sistemas de Numeração **Pesados**:
 - cada algarismo, ocupando uma certa posição num número, é multiplicado por uma potência da base
 - o algarismo atua como coeficiente da potência
 - o valor das potências depende da posição dos dígitos
- a utilização da base como índice permite distinguir números expressos em bases diferentes ($10_{10} \neq 10_2$)
 - sem índice, a base 10 é assumida implicitamente

2.2 Sistemas de Numeração Posicionais

- 5836.47_{10} como soma de produtos (coeficientes $\{0,1,\dots,9\}$ multiplicados por potências de 10):

$$5 \times 10^3 + 8 \times 10^2 + 3 \times 10^1 + 6 \times 10^0 + 4 \times 10^{-1} + 7 \times 10^{-2}$$

- 11001.11_2 como soma de produtos (coeficientes $\{0,1\}$ multiplicados por potências de 2):

$$\begin{aligned} 1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1} + 1 \times 2^{-2} = \\ 16 + 8 + 0 + 0 + 1 + 0.5 + 0.25 = 25.75_{10} \end{aligned}$$

2.3 Conversão entre Sist. de Numeração

- nota: 2.3 aplica-se apenas a **inteiros não-negativos**
- a conversão entre sistemas pode fazer-se usando
 - **método da subtração repetida**
 - é o método mais intuitivo ... mas exige alguma familiarização com as potências da base alvo
 - baseia-se na obtenção da expressão do número como soma de potências da base para a qual se vai converter
 - **método da divisão com resto** (ver slide 18)

2.3 Conversão entre Sist. de Numeração

– Método da Subtração

- **Converter 155_{10} para binário**
 - exprimir 155 como uma soma de potências de 2, em que o peso de cada potência só pode ser 0 ou 1 (alfabeto binário)
 - conversão mais simples !
 - como $2^8 = 256$, então o resultado terá menos de 8 dígitos
 - ora, para a potência $2^7 = 128$, tem-se $1 \times 128 = 128 \leq 155$
 - aproveita-se o **1** e subtrai-se 128 a 155, pelo que sobra **27**

$$\begin{array}{r} 155 \\ - 128 \\ \hline 27 \end{array} = 2^7 \times \mathbf{1}$$

2.3 Conversão entre Sist. de Numeração

– Método da Subtração

- Converter 155_{10} para binário
 - a próxima potência de 2 ($2^6 = 64$) é superior a **27**, pelo que o seu coeficiente será **0** e continua a sobrar **27**
 - a seguinte potência de 2 ($2^5 = 32$) também é > 27 ; o seu coeficiente será **0** e sobra **27**
 - a seguinte potência de 2 ($2^4 = 16$) já é ≤ 27 , logo o seu coeficiente será **1** e sobra **11**

$$\begin{array}{r} 155 \\ - 128 = 2^7 \times 1 \\ \hline 27 \\ - 0 = 2^6 \times 0 \\ \hline 27 \\ - 0 = 2^5 \times 0 \\ \hline 27 \\ - 16 = 2^4 \times 1 \\ \hline 11 \end{array}$$

2.3 Conversão entre Sist. de Numeração

– Método da Subtração

- Converter 155_{10} para binário
 - a seguinte potência de 2 ($2^3 = 8$) é ≤ 11 , donde o seu coeficiente será **1** e sobra **3**
 - a seguinte potência de 2 ($2^2 = 4$) é > 3 , donde o seu coeficiente será **0** e sobra **3**
 - a seguinte potência de 2 ($2^1 = 2$) é ≤ 3 , donde o seu coeficiente será **1** e sobra **1**
 - a seguinte potência de 2 ($2^0 = 1$) é ≤ 1 , donde o seu coeficiente será **1** e sobra **0**

$$\begin{array}{r} 155 \\ - 128 \\ \hline 27 \end{array} = 2^7 \times \mathbf{1}$$

$$\begin{array}{r} \dots \\ 11 \\ - 8 \\ \hline 3 \end{array} = 2^3 \times \mathbf{1}$$
$$\begin{array}{r} 3 \\ - 0 \\ \hline 3 \end{array} = 2^2 \times \mathbf{0}$$
$$\begin{array}{r} 3 \\ - 2 \\ \hline 1 \end{array} = 2^1 \times \mathbf{1}$$
$$\begin{array}{r} 1 \\ - 1 \\ \hline 0 \end{array} = 2^0 \times \mathbf{1}$$

2.3 Conversão entre Sist. de Numeração – Método da Subtração

- Converter 155_{10} para binário
 - o resultado da conversão, lido de **cima para baixo**, é:
 $155_{10} = \mathbf{10011011}_2$
 - na prática, este método corresponde a exprimir o número a converter como uma soma de potências de 2:
 $155 = 2^7 + 2^4 + 2^3 + 2^1 + 2^0$
 - os bits nas posições dadas pelos expoentes acima ficam a 1, e os restantes ficam a 0

155	
- 128	$= 2^7 \times \mathbf{1}$
27	
- 0	$= 2^6 \times \mathbf{0}$
27	
- 0	$= 2^5 \times \mathbf{0}$
27	
- 16	$= 2^4 \times \mathbf{1}$
11	
- 8	$= 2^3 \times \mathbf{1}$
3	
- 0	$= 2^2 \times \mathbf{0}$
3	
- 2	$= 2^1 \times \mathbf{1}$
1	
- 1	$= 2^0 \times \mathbf{1}$
0	

2.3 Conversão entre Sist. de Numeração

– Método da Subtração

- **Converter 190_{10} para a base 3**
 - exprimir 190 como uma soma de potências de 3, em que o peso de cada potência só pode ser 0, 1 ou 2 (alfabeto ternário)
 - como $3^5 = 243$, então o resultado terá menos de 6 dígitos
 - ora, para a potência $3^4 = 81$, tem-se $2 \times 81 = 162 \leq 190$
 - aproveita-se o 2 e subtrai-se 162 a 190, pelo que resta 28

$$\begin{array}{r} 190 \\ - 162 \\ \hline 28 \end{array} = 3^4 \times 2$$

2.3 Conversão entre Sist. de Numeração – Método da Subtração

- Converter 190_{10} para a base 3

- a próxima potência de 3 ainda não usada é $3^3 = 27$
- o coeficiente/dígito 2 não se pode usar pois $2 \times 27 > 28$
- o coeficiente/dígito 1 pode-se usar pois $1 \times 27 \leq 28$; aproveita-se o 1 e subtrai-se 27 a 28, pelo que resta 1
- a próxima potência de 3 ($3^2 = 9$) é demasiado grande, pelo que o seu coeficiente será 0; continua a sobrar 1

$$\begin{array}{r} 190 \\ - 162 \\ \hline 28 \\ - 27 \\ \hline 1 \\ - 0 \\ \hline 1 \end{array} = 3^4 \times 2$$
$$= 3^3 \times 1$$
$$= 3^2 \times 0$$

2.3 Conversão entre Sist. de Numeração

– Método da Subtração

- Converter 190_{10} para a base 3

- a próxima potência de 3 ($3^1 = 3$), é demasiado grande, pelo que o seu coeficiente será **0**; continua a sobrar **1**
- para a última potência de 3 ($3^0 = 1$), o coeficiente/dígito **1** permite ter $1 \times 3^0 = 1 \leq 1$; aproveita-se o **1** e o resto é **0**
- o resultado da conversão, lido de **cima para baixo**, é:

$$190_{10} = \mathbf{21001}_3$$

190	
<u>- 162</u>	$= 3^4 \times 2$
28	
<u>- 27</u>	$= 3^3 \times 1$
1	
<u>- 0</u>	$= 3^2 \times 0$
1	
<u>- 0</u>	$= 3^1 \times 0$
1	
<u>- 1</u>	$= 3^0 \times 1$
0	

2.3 Conversão entre Sist. de Numeração

– Método da Divisão

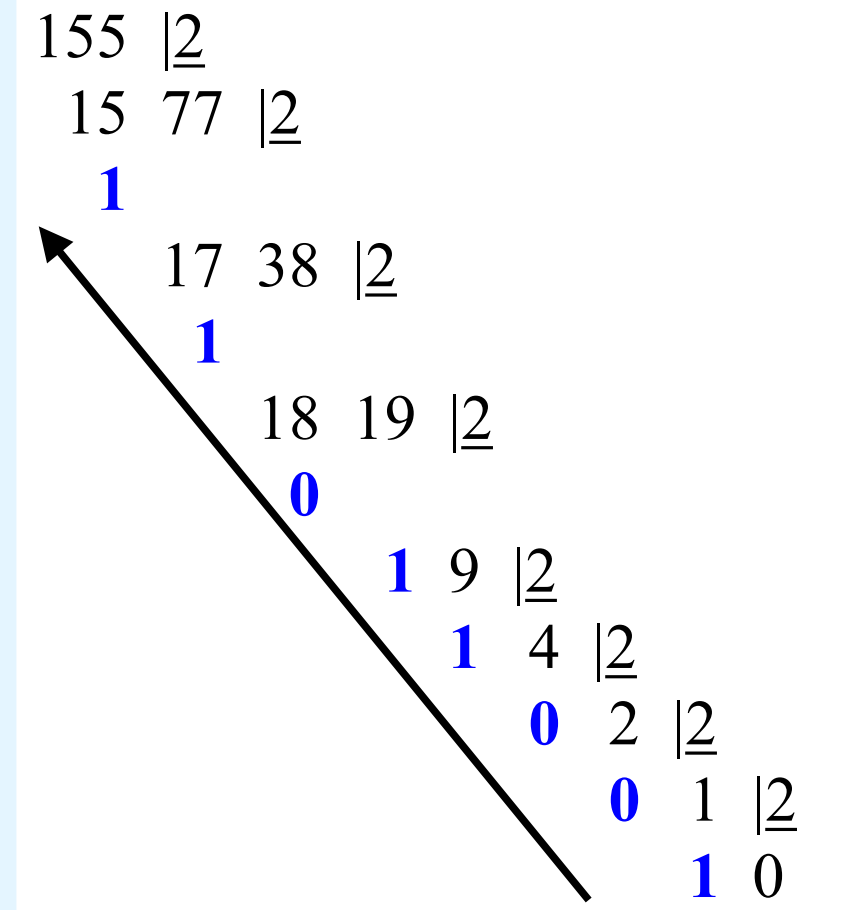
- o **método da divisão com resto** é fácil de mecanizar e dispensa saber as potências da base de destino
- ideia base:
 - sucessivas divisões pelo valor da base equivalem a sucessivas subtrações de potências dessa base
 - os restos obtidos durante o processo de divisão correspondem aos dígitos/coeficientes pretendidos
- de seguida, repetimos as mesmas conversões (155 para a base 2, 190 para a base 3) usando agora este método

2.3 Conversão entre Sist. de Numeração

– Método da Divisão

- **Converter 155_{10} para binário**
 - divisão do número a converter (155), pela base (2) para a qual se pretende converter
 - o processo termina quando o quociente for igual a 0
 - o resultado da conversão, lido de **baixo para cima**, é:

$$155_{10} = \mathbf{10011011}_2$$



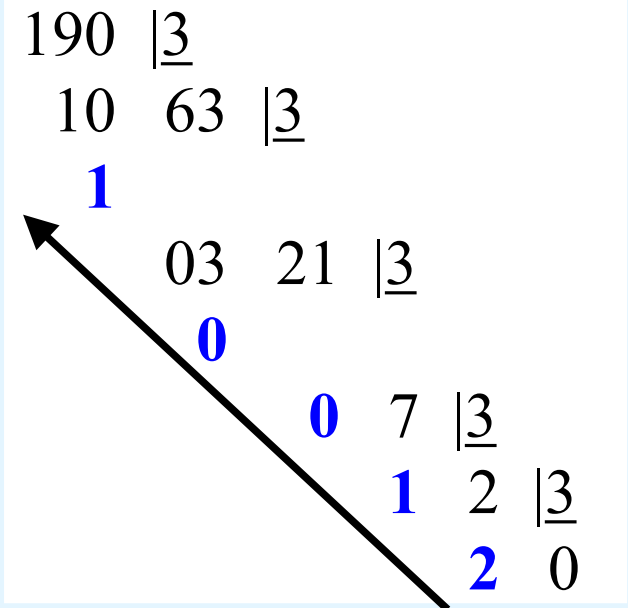
2.3 Conversão entre Sist. de Numeração

– Método da Divisão

- **Converter 190_{10} para a base 3**

- divisão do número a converter (190), pela base (3) para a qual se pretende converter
- o processo termina quando o quociente for igual a 0
- o resultado da conversão, lido de **baixo para cima**, é:

$$190_{10} = \mathbf{21001}_3$$



2.3 Conversão entre Sist. de Numeração

– Conversão de Valores Fracionários

- até aqui, estudou-se a conversão de inteiros (não-negativos); aborda-se agora a conversão de valores fracionários (igual/ não-negativos) ...
- valores fracionários têm uma **parte inteira** e uma **parte fracionária** (por exemplo: **5836.47**)
- ao contrário da parte inteira, a parte fracionária pode não ter representação exata numa base
 - **exemplo:** a quantidade $\frac{1}{2}$ tem representação exata nas bases 2 e 10, mas não na base 3

2.3 Conversão entre Sist. de Numeração

– Conversão de Valores Fracionários

- valores **decimais fracionários** têm dígitos à direita do **ponto decimal**
- valores **fracionários** noutras bases têm dígitos à direita do **ponto da base**
- à direita do *ponto da base*, os dígitos são coeficientes de potências negativas da base em causa:

$$0.47_{10} = 4 \times 10^{-1} + 7 \times 10^{-2}$$

$$0.11_2 = 1 \times 2^{-1} + 1 \times 2^{-2}$$

$$= \frac{1}{2} + \frac{1}{4}$$

$$= 0.5 + 0.25 = 0.75$$

2.3 Conversão entre Sist. de Numeração

– Conversão de Valores Fracionários

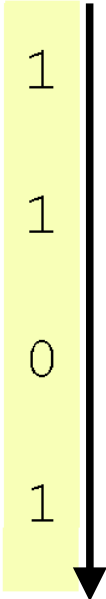
- tal como para valores inteiros, existem dois métodos de conversão: o da **subtração** e o da **multiplicação**
- **método da subtração:**
 - idêntico ao aplicado para valores inteiros, mas ...
 - ... subtraem-se potências negativas da base alvo
 - começa-se pela potência de expoente -1 (n^{-1} , para a base n) e vai-se diminuindo o valor do expoente

2.3 Conversão entre Sist. de Numeração – Método da Subtração (Val. Frac.)

- Conversão de 0.8125_{10} para binário

– o resultado da conversão, lido de **cima para baixo**, é:

$$0.8125_{10} = 0.\mathbf{1101}_2$$

$\begin{array}{r} 0.8125 \\ - 0.5000 \\ \hline 0.3125 \end{array}$	$= 2^{-1} \times 1$	
$\begin{array}{r} 0.3125 \\ - 0.2500 \\ \hline 0.0625 \end{array}$	$= 2^{-2} \times 1$	
$\begin{array}{r} 0.0625 \\ - 0 \\ \hline 0.0625 \end{array}$	$= 2^{-3} \times 0$	
$\begin{array}{r} 0.0625 \\ - 0.0625 \\ \hline 0 \end{array}$	$= 2^{-4} \times 1$	

2.3 Conversão entre Sist. de Numeração

– Método da Subtração (Val. Frac.)

- Este método pode ser usado para converter de decimal para qualquer outra base
- Conversão de 0.4304_{10} para a base 5
 - o resultado da conversão, lido de **cima para baixo**, é:

$$0.4304_{10} = 0.\mathbf{2034}_5$$

0.4304	
- 0.4000	$= 5^{-1} \times \mathbf{2}$
0.0304	
- 0.0000	$= 5^{-2} \times \mathbf{0}$
0.0304	
- 0.0240	$= 5^{-3} \times \mathbf{3}$
0.0064	
- 0.0064	$= 5^{-4} \times \mathbf{4}$
0.0000	



2.3 Conversão entre Sist. de Numeração

– Método da Multiplicação (Val. Frac.)

- **método da multiplicação:**
 - multiplica-se o valor fracionário original pela base
 - no produto, o valor à esquerda do ponto (ou seja, a parte inteira do produto) é o 1º dígito do número
 - o valor à direita do ponto é nova/ multiplicado pela base
 - no produto, o valor à esquerda do ponto é o 2º dígito
 - ...
 - o processo repete-se até que a parte à direita do ponto seja zero (ou até se atingir uma precisão “suficiente”)

2.3 Conversão entre Sist. de Numeração

– Método da Multiplicação (Val. Frac.)

- **Conversão de 0.8125_{10} para binário**

– o primeiro dígito do número pretendido em binário será 1

$$\begin{array}{r} .8125 \\ \times \quad 2 \\ \hline 1.6250 \end{array}$$

2.3 Conversão entre Sist. de Numeração

– Método da Multiplicação (Val. Frac.)

- **Conversão de 0.8125_{10} para binário**
 - ignorando-se o valor das unidades em cada passo, continuar a multiplicar a parte fracionária pelo valor da base

$$\begin{array}{r} .8125 \\ \times \quad 2 \\ \hline 1.6250 \end{array}$$

$$\begin{array}{r} .6250 \\ \times \quad 2 \\ \hline 1.2500 \end{array}$$

$$\begin{array}{r} .2500 \\ \times \quad 2 \\ \hline 0.5000 \end{array}$$

2.3 Conversão entre Sist. de Numeração

– Método da Multiplicação (Val. Frac.)

- **Conversão de 0.8125_{10} para binário**
 - o processo termina quando o produto for zero ou se atingir o número desejado de dígitos
 - o resultado da conversão, lido de **cima para baixo**, é:

$$0.8125_{10} = 0.1101_2$$

	. 8 1 2 5
	× 2
1	. 6 2 5 0
	. 6 2 5 0
	× 2
1	. 2 5 0 0
	. 2 5 0 0
	× 2
0	. 5 0 0 0
	. 5 0 0 0
	× 2
1	. 0 0 0 0

2.3 Conversão entre Sist. de Numeração – Método da Multiplicação (Val. Frac.)

- Este método pode ser usado para converter de decimal para qualquer outra base
- Conversão de 0.4304_{10} para a base 5
 - o resultado da conversão, lido de **cima para baixo**, é:

$$0.4304_{10} = 0.\mathbf{2034}_5$$

	.4304
x	5
	<u>2.1520</u>
	.1520
x	5
	<u>0.7600</u>
	.7600
x	5
	<u>3.8000</u>
	.8000
x	5
	<u>4.0000</u>

2.3 Conversão entre Sist. de Numeração

– Tabela Auxiliar

n	2 ⁿ	3 ⁿ	4 ⁿ	5 ⁿ	6 ⁿ	7 ⁿ	8 ⁿ	9 ⁿ
-5	0,03125	0,0041152263	0,0009765625	0,00032	0,0001286008	0,000059499	3,05176E-005	1,69351E-005
-4	0,0625	0,012345679	0,00390625	0,0016	0,0007716049	0,0004164931	0,0002441406	0,0001524158
-3	0,125	0,037037037	0,015625	0,008	0,0046296296	0,0029154519	0,001953125	0,0013717421
-2	0,25	0,1111111111	0,0625	0,04	0,0277777778	0,0204081633	0,015625	0,012345679
-1	0,5	0,3333333333	0,25	0,2	0,1666666667	0,1428571429	0,125	0,1111111111
0	1	1	1	1	1	1	1	1
1	2	3	4	5	6	7	8	9
2	4	9	16	25	36	49	64	81
3	8	27	64	125	216	343	512	729
4	16	81	256	625	1296	2401	4096	6561
5	32	243	1024	3125	7776	16807	32768	59049
6	64	729	4096	15625	46656	117649	262144	531441
7	128	2187	16384	78125	279936	823543	2097152	4782969
8	256	6561	65536	390625	1679616	5764801	16777216	43046721
9	512	19683	262144	1953125	10077696	40353607	134217728	387420489
10	1024	59049	1048576	9765625	60466176	282475249	1073741824	3486784401

checkpoint:
exercícios 2.1 a 2.4

2.3 Conversão entre Sist. de Numeração

– Conversão expedita entre Bases 2, 8 e 16

- o sistema de numeração **binário** (base 2) é o + importante no mundo dos computadores digitais
 - os números binários são a base da representação de dados em sistemas de computação digitais ...
 - ... e por isso o conhecimento do sistema binário é essencial à compreensão da operação dos componentes de um computador, bem como para o desenho de arquiteturas de conjuntos de instruções

2.3 Conversão entre Sist. de Numeração

– Conversão expedita entre Bases 2, 8 e 16

- no entanto, é difícil ler sequências longas de bits; além disso, um valor decimal da ordem dos milhares já produz uma sequência de bits longa
 - por exemplo: $11010100011011_2 = 13595_{10}$
- por razões de dimensão e facilidade de leitura, valores binários são vulgarmente expressos no sistema de numeração hexadecimal (base 16) ou no sistema de numeração octal (base 8)

2.3 Conversão entre Sist. de Numeração

– Conversão expedita entre Bases 2, 8 e 16

- para converter de **binário para hexadecimal**, basta agrupar os dígitos binários em **grupos de 4 dígitos**
- exemplo: a representação hexadecimal do valor binário 11010100011011_2 ($= 13595_{10}$) é

0011	0101	0001	1011
3	5	1	B

2.3 Conversão entre Sist. de Numeração

– Conversão expedita entre Bases 2, 8 e 16

- para converter de **binário para octal**, é suficiente agrupar os dígitos binários em **grupos de 3 dígitos**
- exemplo: a representação octal do valor binário 11010100011011_2 ($= 13595_{10}$) é

011	010	100	011	011
3	2	4	3	3

2.3.1 Adenda

- Gama de Representação na Base 2 s/ Sinal

- para número **inteiros** binários **sem sinal** (i.e., ≥ 0) expressos com N bits, a gama de representação é:

$$\{ 0, \dots, 2^N - 1 \}$$

- exemplos:

- para N=8 bits

$$\{ 0, \dots, 2^8 - 1 \} = \{ 0, \dots, 255 \}$$

- para N=7 bits

$$\{ 0, \dots, 2^7 - 1 \} = \{ 0, \dots, 127 \}$$

2.3.1 Adenda

- Aritmética Decimal e Aritmética Binária

- a aritmética binária é semelhante à aritmética decimal (nota: veremos apenas exemplos com duas parcelas)

- adição decimal:

- se a soma de uma coluna é ≥ 10 , haverá transporte ...

Transporte			1	1		
		5	6	7	1	9
Parcelas	+	3	1	8	6	3
Soma		8	8	5	8	2

- transporte $\Rightarrow$ somar **1** unidade à próxima coluna

- adição binária:

- regras bit-a-bit $\rightarrow$

$$0 + 0 = 0$$

$$0 + 1 = 1$$

$$1 + 1 = 0$$

com transporte de **1**

$$\Leftrightarrow 1 + 1 = \mathbf{10}$$

- exemplo $\rightarrow$

Transporte	1	1		1	
Parcela		1	1	0	1
		+	1	1	0
				1	1
Soma	1	1	0	1	0

2.3.1 Adenda

- Aritmética Decimal e Aritmética Binária

- subtração decimal:

- numa coluna, se subtraendo > minuendo, haverá empréstimo

- empréstimo => somar **1** ao subtraendo da próx. coluna

Empréstimo		1	1	1	
Minuendo		8	3	0	3
Subtraendo	-	5	4	8	6
Diferença		2	8	1	7

- subtração binária:

- regras bit-a-bit →

0	-	0	=	0
1	-	1	=	0
1	-	0	=	1
0	-	1	=	1 empresta 1

- exemplo →

Empréstimo		1	1		
Minuendo		1	1	0	1
Subtraendo	-	1	1	0	1
Diferença		0	1	1	0

2.4 Representação de Inteiros com Sinal

- até agora, apenas consideramos números ≥ 0
- para representar números inteiros com sinal (+/-), os sistemas de computação reservam o **bit mais significativo** do número para indicar o seu **sinal**
 - bit mais significativo: “o que se encontra mais à esquerda”
- os **restantes bits** representam o **valor** do número

S V V V V V ... V
- as representações c/ sinal mais relevantes são:
 - representação “sinal e magnitude”
 - representação “complemento para um”
 - representação “complemento para dois”

2.4 Representação de Inteiros com Sinal

- Sinal e Magnitude

- bits à direita do bit de sinal: correspondem à representação binária da magnitude/módulo do número
- gama representável com $N = 1 + (N-1)$ bits

$$\{ -(2^{N-1} - 1), \dots, -0, +0, \dots, (2^{N-1} - 1) \}$$

- exemplo: numa palavra de $N=8$ bits, na representação em sinal e magnitude, o valor absoluto é colocado nos $N-1 = 7$ bits menos significativos (à direita do sinal)
 - $+3$ será representado como **00000011**
 - -3 será representado como **10000011**
 - gama representável: $\{ -127, \dots, -0, +0, \dots, 127 \}$

2.4 Representação de Inteiros com Sinal - Complemento para Um

- na base 2, o “complemento para 1” suporta a seguinte representação de inteiros com sinal:
 - **número** > 0 : não é sujeito a qualquer conversão; o valor obtém-se pela leitura direta da representação
 - **número** < 0 : resulta da conversão do módulo desse número para “complemento para 1”; logo:
 - para saber que número > 0 corresponde a um número negativo tem de se calcular o seu complemento para 1
 - exemplo (com números de 8 bits):
 - **+3** será, em complemento para 1: **0**0000011
 - **- 3** será, em complemento para 1: **1**1111100
 - **- 3** será, em “sinal e magnitude”: **1**0000011
- [<https://www.compsilib.com/calculate/ones-complement>]

2.4 Representação de Inteiros com Sinal

- Complemento para Dois

- na base 2, o “complemento para 2” suporta a seguinte representação de inteiros com sinal:
 - **número** >0 : não é sujeito a qualquer conversão; o valor obtém-se pela leitura direta da representação
 - **número** <0 : determinar o complemento para 1 desse número e somar 1
- exemplo (com 8 bits):
 - **+3** é, em complemento para 2: **0**0000011
 - **- 3** é, em complemento para 1: **1**1111100
 - **- 3** é, em complemento para 2: **1**111110**1**

[<https://www.compsclib.com/calculate/twos-complement>]

2.4 Representação de Inteiros com Sinal

- Gamas de Representação (Resumo)

- gama representável com $N = 1 + (N-1)$ bits

- sinal e magnitude, complemento para 1

$$\{ -(2^{N-1} - 1), \dots, -0, +0, \dots, (2^{N-1} - 1) \}$$

- complemento para 2

$$\{ -2^{N-1}, \dots, 0, \dots, (2^{N-1} - 1) \}$$

checkpoint:

exercícios 2.5,
2.6, 2.7 //, 2.16

- exemplo: $N=4$ bits : $\{ -8, \dots, 0, \dots, 7 \}$

- representação dos zeros

- sinal e magnitude: $0\ 0\dots 0 (+0)$; $1\ 0\dots 0 (-0)$

- complemento para 1: $0\ 0\dots 0 (+0)$; $1\ 1\dots 1 (-0)$

- complemento para 2: $0\ 0\dots 0$

2.5 Representação em Vírgula Flutuante

- as representações em “sinal e magnitude”, “complemento para 1” e “complemento para 2” apenas permitem manipular inteiros
- sem modificação, estes formatos não são úteis em aplicações científicas que necessitem de usar valores numéricos reais
- solução: representação em vírgula flutuante

2.5 Representação em Vírgula Flutuante

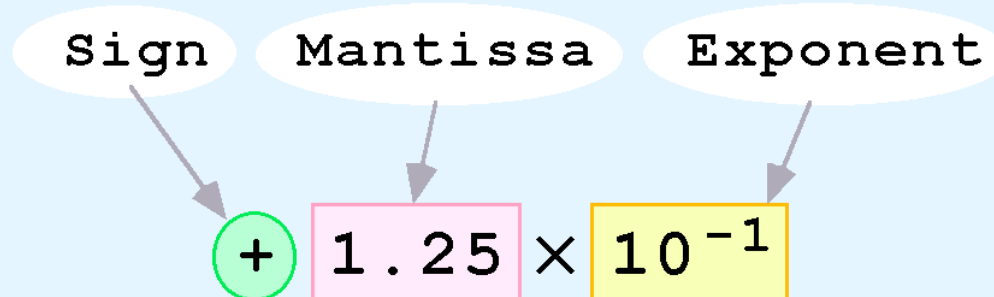
- com algum esforço adicional de programação, pode-se efetuar cálculos em vírgula flutuante usando apenas formatos inteiros
- a isto chama-se *emulação de vírgula flutuante*, visto que os valores em vírgula flutuante não são armazenados como tal; o software faz crer que estão a ser usados valores em vírgula flutuante
- no entanto, a maioria dos computadores atuais possuem hardware dedicado para a realização de cálculos aritméticos em vírgula flutuante

2.5 Representação em Vírgula Flutuante

- valores em vírgula flutuante podem ter um núm. arbitrário de dígitos à direita do ponto decimal
 - por exemplo: $0.5 \times 0.0125 = 0.00625$
- normalmente os valores em vírgula flutuante são representados usando **notação científica**
 - por exemplo:
 - $0.00000000125 = 1.25 \times 10^{-8}$
 - $50000000000000 = 5.0 \times 10^{12}$

2.5 Representação em Vírgula Flutuante

- os computadores usam uma espécie de notação científica para a representação em vírgula flutuante
- em notação científica, os números possuem três componentes: **sinal**, **mantissa** e **expoente** (de 10)



2.5 Representação em Vírgula Flutuante

- num computador, a representação de um número em vírgula flutuante tem três campos de tamanho fixo: **sinal**, **expoente** (de 2) e **significando**



- esta é a disposição *standard* para esses campos

2.5 Representação em Vírgula Flutuante



- o campo do **sinal** corresponde a um único bit
- o tamanho do campo do **expoente** determina a gama de valores que podem ser representados
- o tamanho do campo do **significando** determina a precisão da representação

para ilustração de conceitos será usado um modelo de 14 bits (expoente de 5 bits; significando de 8 bits)

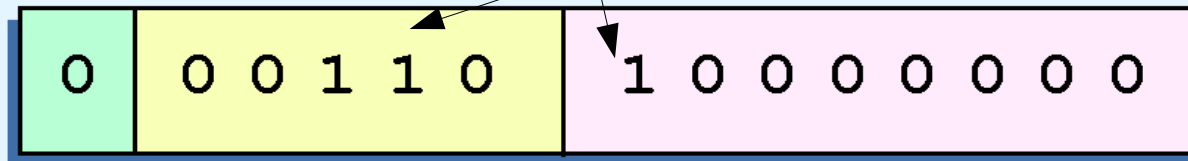
2.5 Representação em Vírgula Flutuante - Modelo de 14 bits



- o significando de um número em vírgula flutuante é sempre precedido por um ponto binário implícito
 - o significando é sempre um valor **fracionário** (<1)
 - para tal, fazer flutuar a vírgula/ponto o necessário, até que o significando seja fracionário
- o expoente indica a potência de 2 que deve ser multiplicada pelo significando

2.5 Representação em Vírgula Flutuante - Modelo de 14 bits

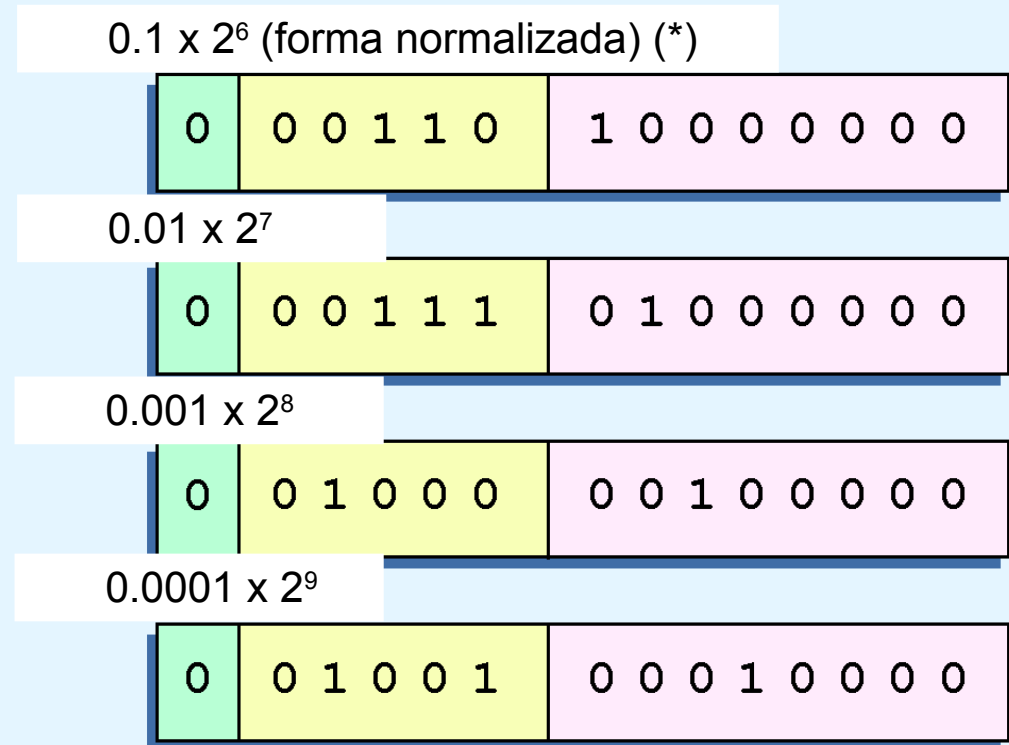
- **exemplo0**: representação de $32_{10} = 100000_2$
 - em **notação científica binária** tem-se $32_{10} = 100000.0_2 \times 2^0 = 10000.00_2 \times 2^1 = \dots = 10.00000_2 \times 2^4 = 1.000000_2 \times 2^5 = 0.1000000_2 \times 2^6 = 0.1_2 \times 2^6$
 - usando esta informação, coloca-se **110₂** (= **6₁₀**) no campo do expoente e **1₂** no campo do significando



2.5 Representação em Vírgula Flutuante - Modelo de 14 bits: Normalização

- **problema:** são possíveis várias representações para o mesmo número
 - desperdício de espaço e fonte de ambiguidade

- **exemplo:** as ilustrações à direita são todas representações possíveis de 32.0



- **solução: normalização** - o 1º bit do significando tem que ser 1, resultando daí um padrão único para cada valor
 - portanto, para o exemplo acima, tem-se $32_{10} = 0.1_2 \times 2^6 (*)$

2.5 Representação em Vírgula Flutuante

- Mod. 14-bits: Expoente com Bias

- **problema:** como representar expoentes **negativos** ?
 - como representar $0.0625_{10} = 0.0001 = 0.1_2 \times 2^{-3}$?
 - notar que não há bit de sinal no campo do expoente !
- **solução1:** no expoente, reservar um bit para o sinal
 - mas há outra solução, mais eficiente em hardware
- **solução2: notação em excesso**
 - os expoentes são guardados como números ≥ 0
 - para obter o verdadeiro valor de um expoente, é necessário subtrair-lhe um **deslocamento (bias)**

2.5 Representação em Vírgula Flutuante

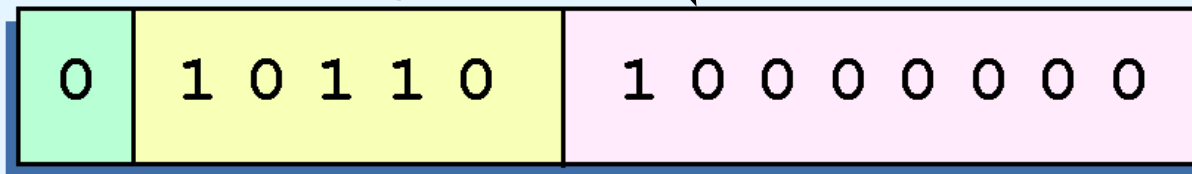
- Mod. 14-bits: Expoente com Bias

- **solução2: notação em excesso (cont.)**
 - o **deslocamento (bias)** é um número que se encontra aproximadamente a meio da gama de números que seria usada para representar expoentes positivos
 - com 5 bits, o expoente pode ter 32 valores (0...31), e portanto usamos um deslocamento de **16**, o que origina uma representação em excesso/bias-16
 - valores menores que o deslocamento representam expoentes negativos, usados p/ números fracionários
 - esta versão revista do modelo de 14-bits será designada como **modelo de 14-bits com bias-16**

2.5 Representação em Vírgula Flutuante

- Modelo de 14-bits com Bias-16

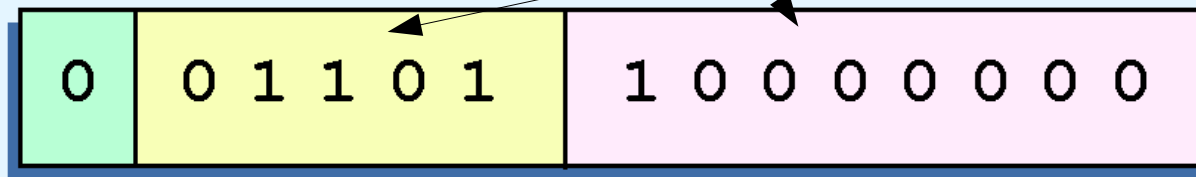
- **exemplo1:** representação de 32_{10}
 - notação científica binária **normalizada**: $32_{10} = 0.1_2 \times 2^6$
 - significando = 10000000_2
 - expoente com bias-16 = $6 + 16 = 22_{10} = 10110_2$
 - graficamente:



2.5 Representação em Vírgula Flutuante

- Modelo de 14-bits com Bias-16

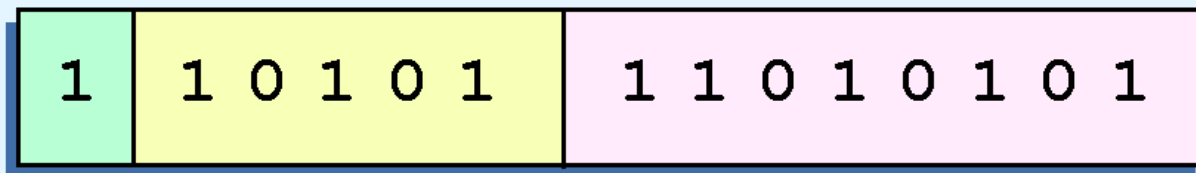
- **exemplo2:** representação de 0.0625_{10}
 - sabemos que 0.0625_{10} é 2^{-4}
 - notação científica binária **normalizada**:
 $0.0625_{10} = 0.0001_2 \times 2^0 = \dots = 0.01_2 \times 2^{-2} = 0.1_2 \times 2^{-3}$
 - significando = 10000000_2
 - expoente com bias-16 = $-3 + 16 = 13_{10} = 01101_2$
 - graficamente:



2.5 Representação em Vírgula Flutuante

- Modelo de 14-bits com Bias-16

- **exemplo3:** representação de -26.625_{10}
 - sabendo-se que $26.625_{10} = 11010.101_2$, após a normalização tem-se $26.625_{10} = 0.11010101_2 \times 2^5$
 - significando = 11010101_2
 - expoente com bias-16 = $5 + 16 = 21_{10} = 10101_2$
 - como $-26.625 < 0$, o bit do sinal será **1**
 - graficamente:



2.5 Representação em Vírgula Flutuante

- Mod. de 14-bits c/ Bias-16: Zeros, Gamas

- **zeros:**

- $|0|00000|000000000| = +0$
- $|1|00000|000000000| = -0$

checkpoint:
exercício 2.18

- **gama de positivos:**

- < positivo: $|0|00000|100000000| = + 0.1 \times 2^{(0-16=-16)} =$
 $+ 0.0000000000000000001 \times 2^0 = (*) 0.000000762939453125$
- > positivo: $|0|11111|111111111| = + 0.111111111 \times 2^{(31-16=15)}$
 $= + 1111111110000000.0 \times 2^0 = (*) 32640$

- **gama de negativos:**

- < negativo: $|1|11111|111111111| = - 32640$
- > negativo: $|1|00000|100000000| = - 0.000000762939453125$

2.5 Representação em Vírgula Flutuante

- Operações Aritméticas

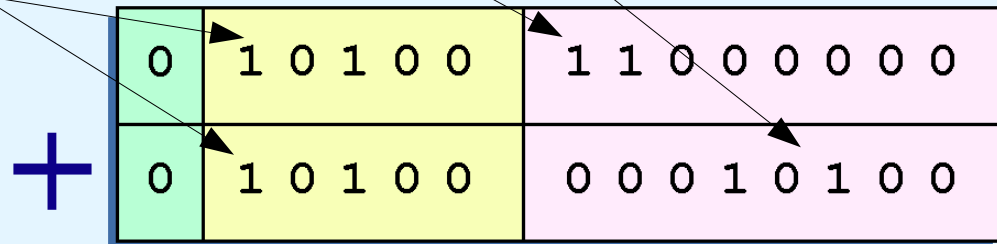
- a adição e a subtração em vírgula flutuante faz-se com métodos análogos aos usados pelos humanos
- inicialmente, expressam-se os dois operandos com o mesmo expoente; se for necessário, desnormaliza-se um dos operandos (o de menor valor absoluto)
- mantendo o expoente, adicionam-se os significandos
- se a soma dos significandos gerar transporte (1) na coluna do bit mais significativo, o transporte entra pela esquerda no significando (que perde o bit menos significativo) e ao expoente soma-se esse transporte

nota: mais adiante introduz-se um método alternativo

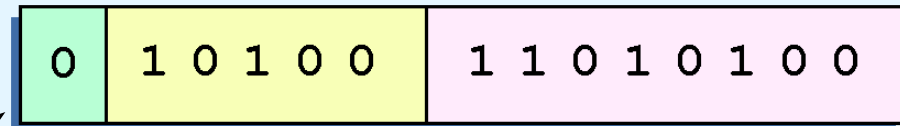
2.5 Representação em Vírgula Flutuante

- Modelo de 14-bits com Bias-16: Adição

- **exemplo** (s/ transporte): somar 12_{10} com 1.25_{10}
 - $12_{10} = 0.1100_2 \times 2^4$
 - $1.25_{10} = 0.101_2 \times 2^1 = 0.000101_2 \times 2^4$
 - expoente = $4 + 16$
= $20 = 10100_2$



- a soma é $0.110101_2 \times 2^4$



2.5 Representação em Vírgula Flutuante

- Modelo de 14-bits com Bias-16: Adição

- **exemplo** (c/ transporte): somar 0.5_{10} a si próprio

- $0.5_{10} = 0.1_2 \times 2^0$

- expoente = $0 + 16 = 16 = 10000_2$

- soma:

$$\begin{array}{r} | 0 | 10000 | 10000000 | \\ + | 0 | 10000 | 10000000 | \\ \hline 10000 | 00000000 | \end{array}$$

- acerto: $ \underline{+1} | 10000000 | 0$

$$| 0 | 10001 | 10000000 | =$$

$$0.1_2 \times 2^{17-16} = 0.1_2 \times 2^1 = 1.0_2 \times 2^0 = 1_{10}$$

checkpoint:
exercícios
2.20,
 $2^{*.1}$ a $2^{*.7}$

2.5 Representação em Vírgula Flutuante

- Modelo de 14-bits com Bias-16: Adição

- **método alternativo:** somar os números desnormalizados (i.e., com expoente 0) e normalizar a soma

– **exemplo** ($12 + 1.25 = 13.25$):

- $12_{10} = 1100.0_2 \times 2^0$

- $1.25_{10} = 1.01_2 \times 2^0$

- soma: 1100.00

$$+ \underline{1.01}$$

$$1101.01 = 1101.01 \times 2^0 = 0.110101 \times 2^4$$

- rep. virg. flutuante: $|0|10100|110101\underline{00}| = 13.25$



$$20 = 16 + 4$$

nota: este método não é adequado para os computadores; estes guardam os números em formato normalizado e só em caso de necessidade (ver método anterior) é que há desnormalização (e de um só número).

2.5 Representação em Vírgula Flutuante

- Modelo de 14-bits com Bias-16: Adição

- **método alternativo** (cont.)

- **exemplo** ($0.5 + 0.5 = 1.0$):

- $0.5_{10} = 0.1_2 \times 2^0$

- soma: **1**

$$0.1$$

$$+ \underline{0.1}$$

$$1.0 = 1.0 \times 2^0 = 0.1 \times 2^1$$

- rep. virg. flutuante: $|0|10001|100000000| = 1.0$



$$17 = 16 + \mathbf{1}$$

2.5 Representação em Vírgula Flutuante

- Modelo de 14-bits com Bias-16: Adição

- **método alternativo** (cont.)

- **exemplo** ($3.125 + 0.6015625 = 3.7265625$):

- $3.125_{10} = 11.001_2 \times 2^0$

- $0.6015625_{10} = 0.100110\mathbf{1}_2 \times 2^0$

- soma: 11.001

$$+ \underline{0.100110\mathbf{1}}$$

$$11.101110\mathbf{1} \times 2^0 = 0.11101110\mathbf{1} \times 2^2$$

- rep. virg. flutuante: $|0|10010|11101110|\mathbf{1} = 3,71875$

↑
 $18 = 16 + \mathbf{2}$

↘ perdeu-se $1/128$

2.5 Representação em Vírgula Flutuante - Norma IEEE-754

- **precisão simples** (32 bits): 1+31 bits
 - 8 bits para o expoente, com bias-127 (2^7-1)
 - 23 bits para o significando
- **precisão dupla** (64 bits): 1+63 bits
 - 11 bits para o expoente, com bias-1023 ($2^{10}-1$)
 - 52 bits para o significando
- na forma normalizada, **o primeiro bit 1 é implícito**
 - i.e., esse bit 1 não é guardado no 1º bit do significando
 - permite **um bit extra (o mais à direita)** para maior precisão
 - exemplo de reconstrução de um número (precisão simples):
 $| \textcolor{blue}{s} | \textcolor{red}{eeeeeeee} | \textcolor{green}{mmmmmmmmmmmmmmmmmmmmmmmmmmmmmmmmmmmm} | =$
 $\textcolor{blue}{s} \textcolor{blue}{1} . \textcolor{green}{mmmmmmmmmmmmmmmmmmmmmmmmmmmmmmmmmmmm} m_2 \times 2^{\textcolor{red}{eeeeeeee}_2 - 127}$

2.5 Representação em Vírgula Flutuante - Norma IEEE-754

- exemplos de precisão simples com representação exata

$$\begin{aligned} & - \quad | 0 \mid 01111111 \mid 000000000000000000000000 \mid = \\ & + 1.000000000000000000000000_2 \times 2^{(01111111_2 - 127)} = \\ & + 1.0_2 \times 2^{(127_{10} - 127_{10})} = + 1.0_2 \times 2^0 = 1.0_{10} \end{aligned}$$

$$\begin{aligned} & - \quad | 0 \mid 01111110 \mid 000000000000000000000000 \mid = \\ & + 1.000000000000000000000000_2 \times 2^{(01111110_2 - 127_{10})} = \\ & + 1.0_2 \times 2^{(126_{10} - 127_{10})} = + 1.0_2 \times 2^{-1} = + .10_2 \times 2^0 = 0.5_{10} \end{aligned}$$

$$\begin{aligned} & - \quad | 0 \mid 10000011 \mid 001110000000000000000000 \mid = \\ & + 1.001110000000000000000000_2 \times 2^{(10000011_2 - 127_{10})} = \\ & + 1.00111_2 \times 2^{(131_{10} - 127_{10})} = + 1.00111_2 \times 2^4 = + 10011.1_2 \times 2^0 = 19.5_{10} \end{aligned}$$

$$\begin{aligned} & - \quad | 1 \mid 10000000 \mid 111000000000000000000000 \mid = \\ & - 1.111000000000000000000000_2 \times 2^{(10000000_2 - 127_{10})} = \\ & - 1.111_2 \times 2^{(128_{10} - 127_{10})} = - 1.111_2 \times 2^1 = - 11.11_2 \times 2^0 = -3.75_{10} \end{aligned}$$

2.5 Representação em Vírgula Flutuante - Norma IEEE-754

- exemplo de precisão simples sem representação exata

$$\begin{aligned} & - \quad | 0 | \textcolor{red}{01111011} | \textcolor{green}{10011001100110011001101} | = \\ & + 1.\textcolor{green}{10011001100110011001101}_2 \times 2^{(\textcolor{red}{01111011}_2 - 127)} = \\ & + 1.\textcolor{green}{10011001100110011001101}_2 \times 2^{(\textcolor{red}{123}_{10} - 127_{10} = -4)} = \\ & + 0.000\textcolor{purple}{1}\textcolor{green}{0011001100110011001101}_2 \times 2^0 (*) = \end{aligned}$$

0.100000001490116119384765625₁₀ = rep. mais próxima possível de 0.1

- para confirmar este valor:  erro = 0.000000001490116119384765625
 - calcular a soma de produtos correspondente a (*) - ver slide seguinte
 - usar <https://www.exploringbinary.com/floating-point-converter/>
 - imprimir 0.1 como `float` num programa em C:

```
#include <stdio.h>
int main() {
    float x=0.1;
    printf("%.30f\n", x); // precision of 30 digits with right zeros
    printf("%.30g\n", x); // precision of 30 digits without right zeros
}
```

```
0.100000001490116119384765625000
0.100000001490116119384765625
```

2.5 Representação em Vírgula Flutuante

- Norma IEEE-754

- exemplo de precisão simples sem representação exata
 - conversão de $0.000110011001100110011001101_2$ para a base 10

A	B	C	D
exponent	power of 2	bit	(power of 2) x (bit)
-1	0,5	0	0
-2	0,25	0	0
-3	0,125	0	0
-4	0,0625	1	0,0625
-5	0,03125	1	0,03125
-6	0,015625	0	0
-7	0,0078125	0	0
-8	0,00390625	1	0,00390625
-9	0,001953125	1	0,001953125
-10	0,0009765625	0	0
-11	0,00048828125	0	0
-12	0,000244140625	1	0,000244140625
-13	0,0001220703125	1	0,0001220703125
-14	6,103515625E-05	0	0
-15	3,0517578125E-05	0	0
-16	1,52587890625E-05	1	1,52587890625E-05
-17	7,62939453125E-06	1	7,62939453125E-06
-18	3,814697265625E-06	0	0
-19	1,9073486328125E-06	0	0
-20	9,5367431640625E-07	1	9,5367431640625E-07
-21	4,76837158203125E-07	1	4,76837158203125E-07
-22	2,38418579101563E-07	0	0
-23	1,19209289550781E-07	0	0
-24	5,96046447753906E-08	1	5,960464477539063E-08
-25	2,98023223876953E-08	1	2,9802322387695313E-08
-26	1,49011611938477E-08	0	0
-27	7,45058059692383E-09	1	7,450580596923828E-09
sum of products:			0,100000001490116119384765625000

Nota: cálculo feito na aplicação Gnumeric, que mostra todos os dígitos do resultado; o Microsoft Excel e LibreOffice só mostram o valor 0,100000001490116 (mas internamente preservam o valor correto)

2.5 Representação em Vírgula Flutuante

- Norma IEEE-754

- exemplo de precisão dupla sem representação exata

– $|0|0111111011|100110011001100110011001100110011001100110011010| =$
 $+ 1.100110011001100110011001100110011001100110011010_2 \times 2^{(0111111011)_2 - 1023} =$
 $+ 1.100110011001100110011001100110011001100110011010_2 \times 2^{(1019)_{10} - 1023_{10} = -4} =$
 $+ 0.000110011001100110011001100110011001100110011010_2 \times 2^0 = \dots =$
 $0.10000000000000000055511151231257827021181583404541015625_{10} \approx 0.1$

– para confirmar este valor:

erro = 0.0000000000000000005551115...

- usar <https://www.exploringbinary.com/floating-point-converter/>
- imprimir 0.1 como double num programa em C:

```
#include <stdio.h>
int main() {
    double xx=0.1;
    printf("%.60f\n", xx); // precision of 60 digits with right zeros
    printf("%.60g\n", xx); // precision of 60 digits without right zeros
    printf("%.60g\n", 0.1); // note the implicit conversion of 0.1 to double
}
```

```
0.1000000000000000005551115123125782702118158340454101562500000
0.10000000000000000055511151231257827021181583404541015625
0.10000000000000000055511151231257827021181583404541015625
```

2.5 Representação em Vírgula Flutuante - Norma IEEE-754

- **zeros**
 - só zeros no expoente e mantissa: $|0/1| \mathbf{0} \dots \mathbf{0} | \mathbf{0} \dots \mathbf{0} |$
 - há 2 zeros (+0 e -0); comparar zeros pode retornar Falso !
 - além disso, neste caso o bit 1 implícito não é usado
- **valores desnormalizados**
 - $|0/1| \mathbf{0} \dots \mathbf{0} | \text{non-zero} |$; sem bit implícito
 - para representar números próximos de zero
- **valores especiais**
 - só uns no expoente: $|0/1| \mathbf{1} \dots \mathbf{1} | x \dots x |$
 - mantissa só com zeros: +/- infinito
 - mantissa diferente de zero: NaN (not-a-number)

2.5 Representação em Vírgula Flutuante

- Erros de Representação

- independentemente do número de bits usado na representação em vírgula flutuante, o modelo em causa tem uma capacidade finita de representação
- o universo de números reais é infinito; portanto, qualquer modelo de representação oferecerá apenas aproximações dos valores reais
- em algum ponto, todos os modelos acabam por falhar introduzindo erros nos cálculos
- aumentando o número de bits do modelo, reduzem-se esses erros, mas nunca se eliminam por completo

2.5 Representação em Vírgula Flutuante

- Erros de Representação

- o objetivo é reduzir os erros ou, pelo menos, ter alguma ideia da magnitude dos erros nos cálculos
- **exemplo:**
 - no modelo de 14 bits, não é possível representar com exatidão o valor decimal 128.5, pois seriam necessários 9 bits no significando: $128.5_{10} = 10000000.1_2$
 - quando se tenta representar 128.5_{10} no modelo de 14 bits, perde-se o **bit menos significativo** e produz-se um erro relativo de $(128.5 - 128) / 128.5 = 0.39\%$
- é ainda necessário levar em conta que os erros se podem acumular ao longo de repetidas operações

2.5 Representação em Vírgula Flutuante - Erros de Representação

- **exemplo:** se, repetidamente, se adicionasse 0.5 a 128.5, haveria um erro de 2% ao fim de 4 iterações
 - $[(128.5+0.5+0.5+0.5+0.5) - (128)] / 130.5 = 1.9\%$
- os erros podem ser minimizados quando se usam operandos com magnitudes similares ou próximas:
 - para adicionar repetidamente 0.5 a 128.5, será melhor adicionar 0.5 a si próprio e adicionar o resultado a 128.5
 - $0.5+0.5+0.5+0.5=2$; $2 + 128.5 = 130$ (perde-se 0.5, logo o erro relativo seria $(130.5 - 130)/130.5 = 0.38\%$)
- nestes exemplos, o erro surge pela perda do bit (-) significativo; a perda do (+) significativo seria pior !

2.5 Representação em Vírgula Flutuante - Erros de Representação

- Devido à truncagem de bits, nem sempre se pode assumir que uma operação em vírgula flutuante é comutativa ou distributiva, ou seja, nem sempre

$$(a + b) + c = a + (b + c)$$

$$a*(b + c) = ab + ac$$

- Para testar se um valor em virgula flutuante é igual a outro, pode-se usar um grau de aproximação *epsilon*:

if (abs(x-y)) < epsilon) **then** ...

uante

- Os seguintes programas ilustram alguns casos problemáticos

```
#include <stdio.h>
#include <math.h>

int main()
{
    float x=0.1, y=0.2, z=0.3, w=0.4, a, b;

    a=y-x; b=w-z; // a == b == 0.1, right ?
    if (a!=b) {    // unfortunately, no !
        printf("%.30g\n", a);
        printf("%.30g\n", b);
    }
    printf("error: %.30g\n", fabs(b-a));
}
```

```
a: 0.100000001490116119384765625
b: 0.0999999940395355224609375
error: 7.450580596923828125e-09
```

```
#include <stdio.h>
#include <math.h>
// #define EPSILON 1.0e-16
#define EPSILON 1.0e-3

int main()
{
    double a=0.1, b=0.2, c=0.3, d, e, error;

    d=(a+b)+c; e=a+(b+c); error=fabs(d-e);
    printf("d: %.60g\n", d); printf("e: %.60g\n", e);
    printf("error: %.60g\n", error);
    if (error < EPSILON) printf("values considered equal\n");
    else printf("values considered different\n");
}
```

```
d:0.600000000000000088817841970012523233890533447265625  
e:0.59999999999999997779553950749686919152736663818359375  
error:1.1102230246251565404236316680908203125e-16  
values considered equal
```

2.5 Representação em Vírgula Flutuante - Erros de Representação

- **Um caso real (*):**
 - o seguinte programa estava a ser usado por um odómetro para incrementar um contador (metros) por cada 0,01m (1 cm) percorridos; mas ao cabo de 10 000 Km, o resultado é completamente diferente do que devia ser ! porquê ?

```
#include <stdio.h>

int main() {
    float meters = 0;
    int iterations = 1000000000; // 10000Km / 0,01m = 1000000000
    for (int i = 0; i < iterations; i++) meters += (float)0.01;
    printf("Expected: %.30g meters\n", (float)0.01*iterations);
    printf("Obtained: %.30g meters\n", meters);
}
```

```
Expected: 10000000 meters
Obtained: 262144 meters
```

Após atingir 262144, todos os incrementos realizados deixam de ter qualquer efeito !

2.5 Representação em Vírgula Flutuante

- Erros de Representação

- **Um caso real: explicação**

- em IEEE-754 só certos números são representáveis de forma exata, e a distância entre esses números tende a aumentar
- assim, um número x é representado pelo número y mais próximo dele que é representável de forma exata; e, por omissão, a escolha de y decide-se por arredondamento de x
- a seguir a $x=262144.0$ o próximo número representável é $y=262144.03125$ [e os seguintes estão à mesma distância entre si, de 0.03125, até se alcançar 524287.96875, altura em que a distância entre os números passa para o dobro (ver (*))]
- ora, $x + 0.01 = 262144.01$; mas para que este número fosse convertido para $y=262144.03125$, em vez de $x+0.01$ teríamos que ter pelo menos $x + (0.03125/2) = x + 0.015625 \dots$

(*) <https://float.exposed/0x48800000>

2.5 Representação em Vírgula Flutuante

- Erros de Representação

- Um caso real: soluções

```
#include <stdio.h>
#include <math.h>

int main() {
    double meters = 0;
    int iterations = 1000000000;
    for (int i = 0; i < iterations; i++) meters += (double)0.01;
    printf("Expected: %.30g meters\n", (double)0.01*iterations);
    printf("Obtained: %.30g meters\n", meters);
    printf("Error: %.30g meters\n", fabs((double)0.01*iterations-meters));
}
```

Expected: 10000000 meters
Obtained: 9999999.82515866868197917938232 meters
Error: 0.17484133131802082061767578125 meters

Ainda não igual, mas erro muito menor.

```
#include <stdio.h>
#include <math.h>

int main() {
    double meters = 0;
    int iterations = 1000000000 / 50;
    for (int i = 0; i < iterations; i++) meters += (double)0.5;
    printf("Expected: %.30g meters\n", (double)0.5*iterations);
    printf("Obtained: %.30g meters\n", meters);
    printf("Error: %.30g meters\n", fabs((double)0.5*iterations-meters));
}
```

Expected: 10000000 meters
Obtained: 10000000 meters
Error: 0 meters

Mudando a unidade mínima de incremento, conseguiu-se eliminar o erro.

2.5 Representação em Vírgula Flutuante

- Erros de Representação

- o *overflow* e o *underflow* em vírgula flutuante podem levar os programas a terminar abruptamente
- ocorre *overflow* quando não há espaço para guardar os bits + significativos que resultam de um cálculo
- ocorre *underflow* quando um número é demasiado pequeno para ser armazenado; uma situação deste tipo pode resultar numa tentativa de divisão por zero

É melhor que um programa termine abruptamente, ou que produza resultados incorretos ?

2.6 Códigos de Caracteres



- os valores numéricos codificados em binário terão pouca utilidade se não forem apresentados de forma perceptível aos utilizadores
- assim, os “caracteres compreendidos pelos humanos” têm de ser convertidos para “padrões de bits compreendidos pelos computadores” usando esquemas de codificação de caracteres
- os códigos de caracteres evoluíram progressiva/
 - memórias e sistemas de armazenamento com maior capacidade permitiram códigos mais elaborados

2.6 Códigos de Caracteres

- BCD e EBCDIC



- um dos 1ºs códigos foi o **BCD** (*Binary-Coded Decimal*), usado pela IBM nos anos 50 e 60
 - uma variante, de 6 bits, permitia representar dígitos decimais e apenas letras maiúsculas
- em 1964, o BCD foi aumentado para 8 bits, passando a chamar-se **EBCDIC** (*Extended Binary-Coded Decimal Interchange Code*)
 - o EBCDIC foi um dos primeiros códigos amplamente utilizados, que suportava minúsculas e maiúsculas, para além de caracteres de pontuação e de controlo
 - o EBCDIC e o BCD são atualmente ainda usados em sistemas de grande porte (mainframes) da IBM

2.6 Códigos de Caracteres - EBCDIC

Zone	Digit															
	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
0000	NUL	SOH	STX	ETX	PF	HT	LC	DEL		RLF	SMM	VT	FF	CR	SR	SI
0001	DLE	DC1	DC2	TM	RES	NL	BS	IL	CAN	EM	CC	CU1	IFS	IGS	IRS	IUS
0010	DS	SOS	FS		BYP	LF	ETB	ESC			SM	CU2		ENQ	ACK	BEL
0011			SYN		PN	RS	UC	EOT				CU3	DC4	NAK		SUB
0100	SP										[	.	<	(	+	!
0101	&										]	\$	*	)	;	^
0110	-	/									!	,	%	_	>	?
0111										'	:	#	@	'	=	"
1000		a	b	c	d	e	f	g	h	i						
1001		j	k	l	m	n	o	p	q	r						
1010		~	s	t	u	v	w	x	y	z						
1011																
1100	{	A	B	C	D	E	F	G	H	I						
1101	}	J	K	L	M	N	O	P	Q	R						
1110	\		S	T	U	V	W	X	Y	Z						
1111	0	1	2	3	4	5	6	7	8	9						

- facilidade de conversão minúsculas ↔ maiúsculas:
 - basta negar o 2º bit mais significativo ...

2.6 Códigos de Caracteres - EBCDIC



NUL	Null	TM	Tape mark	ETB	End of transmission block
SOH	Start of heading	RES	Restore	ESC	Escape
STX	Start of text	NL	New line	SM	Set mode
ETX	End of text	BS	Backspace	CU2	Customer use 2
PF	Punch off	IL	Idle	ENQ	Enquiry
HT	Horizontal tab	CAN	Cancel	ACK	Acknowledge
LC	Lowercase	EM	End of medium	BEL	Ring the bell (beep)
DEL	Delete	CC	Cursor Control	SYN	Synchronous idle
RLF	Reverse linefeed	CU1	Customer use 1	PN	Punch on
SMM	Start manual message	IFS	Interchange file separator	RS	Record separator
VT	Vertical tab	IGS	Interchange group separator	UC	Uppercase
FF	Form Feed	IRS	Interchange record separator	EOT	End of transmission
CR	Carriage return	IUS	Interchange unit separator	CU3	Customer use 3
SO	Shift out	DS	Digit select	DC4	Device control 4
SI	Shift in	SOS	Start of significance	NAK	Negative acknowledgement
DLE	Data link escape	FS	Field separator	SUB	Substitute
DC1	Device control 1	BYP	Bypass	SP	Space
DC2	Device control 2	LF	Line feed		

- nos caracteres especiais reconhecem-se alguns ainda usados hoje em dia (NUL, DEL, CR, NL, BS, ESC, SP) e outros já obsoletos (PF e PN, para cartões perfurados)

2.6 Códigos de Caracteres - ASCII



- outros fabricantes, mais preocupados com normas de transmissão de dados entre sistemas, adotaram o **ASCII** (*American Standard Code for Information Interchange*), com 7 bits, em substituição dos códigos proprietários de 6 e 8 bits
- enquanto o BCD e o EBCDIC são herdeiros de códigos usados em cartões perfurados, o ASCII derivou de códigos de telecomunicações (telex)
- até recentemente, o código ASCII foi o código dominante fora do mundo dos mainframes IBM

2.6 Códigos de Caracteres - ASCII



0	NUL	16	DLE	32		48	0	64	@	80	P	96	`	112	p
1	SOH	17	DC1	33	!	49	1	65	A	81	Q	97	a	113	q
2	STX	18	DC2	34	"	50	2	66	B	82	R	98	b	114	r
3	ETX	19	DC3	35	#	51	3	67	C	83	S	99	c	115	s
4	EOT	20	DC4	36	\$	52	4	68	D	84	T	100	d	116	t
5	ENQ	21	NAK	37	%	53	5	69	E	85	U	101	e	117	u
6	ACK	22	SYN	38	&	54	6	70	F	86	V	102	f	118	v
7	BEL	23	ETB	39	'	55	7	71	G	87	W	103	g	119	w
8	BS	24	CAN	40	(	56	8	72	H	88	X	104	h	120	x
9	TAB	25	EM	41	)	57	9	73	I	89	Y	105	i	121	y
10	LF	26	SUB	42	*	58	:	74	J	90	Z	106	j	122	z
11	VT	27	ESC	43	+	59	;	75	K	91	[	107	k	123	{
12	FF	28	FS	44	,	60	<	76	L	92	\	108	l	124	
13	CR	29	GS	45	-	61	=	77	M	93	]	109	m	125	}
14	SO	30	RS	46	.	62	>	78	N	94	^	110	n	126	~
15	SI	31	US	47	/	63	?	79	O	95	_	111	o	127	DEL

NUL	Null	DLE	Data link escape
SOH	Start of heading	DC1	Device control 1
STX	Start of text	DC2	Device control 2
ETX	End of text	DC3	Device control 3
EOT	End of transmission	DC4	Device control 4
ENQ	Enquiry	NAK	Negative acknowledge
ACK	Acknowledge	SYN	Synchronous idle
BEL	Bell (beep)	ETB	End of transmission block
BS	Backspace	CAN	Cancel
HT	Horizontal tab	EM	End of medium
LF	Line feed, new line	SUB	Substitute
VT	Vertical tab	ESC	Escape
FF	Form feed, new page	FS	File separator
CR	Carriage return	GS	Group separator
SO	Shift out	RS	Record separator
SI	Shift in	US	Unit separator
		DEL	Delete/Idle

- 32 caracteres de controlo (NUL, SOH, ...), 10 dígitos (0-9), 52 letras (a-zA-Z), 32 caracteres especiais (\$, #, ...), espaço
- 8º bit usável como **bit de paridade**, para deteção de erros
 - mas utilidade cada vez <, pela > fiabilidade dos computadores
 - 8º bit é usado para codificar mais caracteres especiais (ç,ã,...)

2.6 Códigos de Caracteres

- Bit(s) de Paridade



- a forma mais básica de deteção de erros
- bit de **paridade par** (*even parity*):
 - 1 se (“número de 1s do conjunto” + 1) for **par**
- bit de **paridade ímpar** (*odd parity*):
 - 1 se (“número de 1s do conjunto” + 1) for **ímpar**

7 bits de dados (número de 1s)	8 bits (incluindo bit de paridade)	
	paridade PAR	paridade ÍMPAR
0000000 (0)	0 0000000	1 0000000
1010001 (3)	1 1010001	0 1010001
1101001 (4)	0 1101001	1 1101001
1111111 (7)	1 1111111	0 1111111

2.6 Códigos de Caracteres - Unicode



- EBCDIC e ASCII assentam no alfabeto latino, usado por uma minoria da população mundial
- muitos dos sistemas atuais optaram pelo Unicode, um sistema de 16 bits que permite codificar os caracteres de qualquer linguagem
 - a linguagem Java e alguns sistemas operativos usam o Unicode como código de caracteres por omissão
- o espaço de codificação do Unicode divide-se em 6 partes; a primeira parte destina-se aos alfabetos ocidentais, incluindo Latino, Grego e Cirílico

2.6 Códigos de Caracteres - Unicode

- apresenta-se à direita o espaço de codificação Unicode →
 - os caracteres Unicode de numeração + baixa incluem o código ASCII
 - os de numeração + alta permitem a definição de códigos próprios

checkpoint:

exercícios 2.24 a 2.26

Character Types	Language	Number of Characters	Hexadecimal Values
Alphabets	Latin, Greek, Cyrillic, etc.	8192	0000 to 1FFF
Symbols	Dingbats, Mathematical, etc.	4096	2000 to 2FFF
CJK	Chinese, Japanese, and Korean phonetic symbols and punctuation.	4096	3000 to 3FFF
Han	Unified Chinese, Japanese, and Korean	40,960	4000 to DFFF
	Han Expansion	4096	E000 to EFFF
User Defined		4095	F000 to FFFE

Conclusões

- os computadores armazenam dados na forma de bits, bytes e palavras, usando o sistema binário
- os inteiros com sinal podem ser guardados em sinal e magnitude, complemento para 1 e complemento para 2
- números em vírgula flutuante são codificados na norma IEEE 754
- os caracteres são codificados em ASCII, EBCDIC ou Unicode

Referências

- livro ECOA – Capítulo 2
- http://en.wikipedia.org/wiki/Binary_numeral_system
- <http://en.wikipedia.org/wiki/Octal>
- <http://en.wikipedia.org/wiki/Hexadecimal>
- http://en.wikipedia.org/wiki/Signed_number_representations
- http://en.wikipedia.org/wiki/One%27s_complement
- http://en.wikipedia.org/wiki/Two%27s_complement
- http://en.wikipedia.org/wiki/Floating-point_arithmetic
- http://en.wikipedia.org/wiki/IEEE_754-2008
- <http://en.wikipedia.org/wiki/Unicode>
- <http://en.wikipedia.org/wiki/Ascii>
- http://en.wikipedia.org/wiki/Binary-coded_decimal
- <http://en.wikipedia.org/wiki/Ebcdic>